

FUNDAÇÃO EDSON QUEIROZ
UNIVERSIDADE DE FORTALEZA — UNIFOR
CENTRO DE CIÊNCIAS TECNOLÓGICAS
CURSO DE CIÊNCIA DA COMPUTAÇÃO

FLEET MANAGER

Sistema de Gestão Inteligente de Frotas

André Luiz Cavalcante da Silva (2310287)
Luiz Eduardo Pacheco (2310314)
Gregory Figueiredo de M. P. Jereissati (2320425)

Fortaleza — CE

2026

André Luiz Cavalcante da Silva
Luiz Eduardo Pacheco
Gregory Figueiredo de M. P. Jereissati

FLEET MANAGER

Sistema de Gestão Inteligente de Frotas

Trabalho apresentado ao Curso de Ciência da Computação do Centro de Ciências Tecnológicas da Universidade de Fortaleza, como requisito do Projeto Final Integrador I.

Orientador: Prof. Me. Ronaldo Gonçalves Junior

Fortaleza — CE

2026

RESUMO

A gestão de frotas em organizações de pequeno e médio porte costuma ser conduzida de forma descentralizada, distribuída entre planilhas isoladas e controles manuais. A informação existe, porém fragmentada, e o acompanhamento de prazos é passivo: documentos vencem sem que ninguém seja avisado e o custo de cada veículo não é apurado. Este trabalho apresenta o Fleet Manager, aplicação web que centraliza o controle operacional, financeiro e documental de frotas em uma base de dados única, organizada em torno do veículo. O sistema foi construído em TypeScript no servidor e na interface, em um monorepo gerenciado por npm workspaces: a interface é uma Single Page Application em React com Vite, e a interface de programação de aplicações REST, em Node.js com Express, adota arquitetura em camadas com dependência unidirecional entre rotas, controladores, serviços e repositórios. A persistência ocorre em PostgreSQL gerenciado pelo Supabase, acessado exclusivamente por meio do mapeador objeto-relacional Prisma. A autenticação é delegada ao Supabase Auth, que emite tokens assinados com chave assimétrica pelo algoritmo ES256; a aplicação os verifica com a chave pública publicada pelo próprio serviço, sem armazenar segredo algum. O controle de acesso baseado em papéis permanece sob responsabilidade da aplicação, com papel e situação de aprovação persistidos fora do token, o que confere efeito imediato ao bloqueio de contas. A qualidade foi verificada por cem testes automatizados das regras de negócio. O sistema encontra-se publicado em ambiente de produção, com interface e servidor atendidos pelo mesmo domínio. Registra-se, como delimitação, que a validação empregou base de dados fictícia, não tendo o sistema sido aplicado em uma organização real.

Palavras-chave: Gestão de frotas; Arquitetura de software; Aplicações web; Controle de acesso baseado em papéis; Computação em nuvem.

LISTA DE ILUSTRAÇÕES

- Figura 1 — Visão geral da solução
- Figura 2 — Casos de uso significativos para a arquitetura
- Figura 3 — Camadas da aplicação e sentido das dependências
- Figura 4 — Organização em pacotes do monorepo
- Figura 5 — Modelo entidade-relacionamento
- Figura 6 — Autenticação e acesso a recurso protegido
- Figura 7 — Anexo de arquivo ao documento
- Figura 8 — Diagrama de implantação
- Figura 9 — Ciclo de vida da conta de usuário

LISTA DE TABELAS

Tabela 1 — Tecnologias adotadas por camada

Tabela 2 — Requisitos e restrições arquiteturais

Tabela 3 — Requisitos funcionais e situação de atendimento

Tabela 4 — Requisitos não funcionais e situação de atendimento

Tabela 5 — Regras de negócio implementadas

Tabela 6 — Matriz de controle de acesso por papel

Tabela 7 — Entidades do modelo de dados

Tabela 8 — Verificações executadas sobre o código

Tabela 9 — Cronograma resumido

Tabela 10 — Partes interessadas do projeto

LISTA DE ABREVIATURAS E SIGLAS

API	Application Programming Interface
CNH	Carteira Nacional de Habilitação
CRLV	Certificado de Registro e Licenciamento de Veículo
CRUD	Create, Read, Update, Delete
DTO	Data Transfer Object
ES256	ECDSA sobre a curva P-256 com SHA-256
HTTP	HyperText Transfer Protocol
IPVA	Imposto sobre a Propriedade de Veículos Automotores
JWKS	JSON Web Key Set
JWT	JSON Web Token
MVP	Minimum Viable Product
ORM	Object-Relational Mapping
RBAC	Role-Based Access Control
REST	Representational State Transfer
RF	Requisito Funcional
RLS	Row Level Security
RN	Regra de Negócio
RNF	Requisito Não Funcional
SPA	Single Page Application
SQL	Structured Query Language
TLS	Transport Layer Security
UML	Unified Modeling Language

SUMÁRIO

1 INTRODUÇÃO.....	9
1.1 Justificativa.....	9
1.2 Objetivo geral	10
1.3 Objetivos específicos.....	10
1.4 Estrutura do trabalho	10
2 FUNDAMENTAÇÃO TEÓRICA	12
2.1 Gestão de frotas	12
2.2 Aplicações web e arquitetura cliente-servidor.....	12
2.3 Arquitetura REST e interface de programação.....	13
2.4 Banco de dados relacional e mapeamento objeto-relacional.....	13
2.5 Autenticação delegada e controle de acesso.....	13
2.6 Validação de dados e qualidade de software	14
3 METODOLOGIA.....	15
3.1 Ciclo de desenvolvimento	15
3.2 Versionamento e modelagem	15
3.3 Ferramentas e ambiente	15
4 DESENVOLVIMENTO.....	17
4.1 Casos de uso	17
4.2 Arquitetura em camadas	18
4.3 Organização do repositório.....	19
4.4 Modelo de dados.....	20
4.5 Autenticação e controle de acesso	22
4.6 Módulos funcionais	23
4.7 Armazenamento de arquivos	24
4.8 Rotina automatizada de vencimentos	24
4.9 Proteção do banco de dados.....	25

4.10 Publicação em produção	25
4.11 Delimitação do escopo.....	26
5 RESULTADOS E DISCUSSÕES.....	28
5.1 Requisitos atendidos	28
5.2 Verificação da qualidade	29
5.3 Limitações e discussão	30
6 CONCLUSÃO.....	31
REFERÊNCIAS	32
ANEXO A — TERMO DE ABERTURA DO PROJETO	34
A.1 Identificação do projeto	34
A.2 Objetivos.....	34
A.3 Escopo	34
A.4 Cronograma resumido	34
A.5 Partes interessadas	35
A.6 Restrições.....	35
A.7 Riscos e mitigação	35
A.8 Critérios de sucesso	35
ANEXO B — DOCUMENTO DE ARQUITETURA DE SOFTWARE	37
B.1 Requisitos e restrições arquiteturais.....	37
B.2 Visão de casos de uso	37
B.3 Visão lógica	37
B.4 Visão de implementação	37
B.5 Visão de processo	38
B.6 Visão de implantação	38
B.7 Visão de dados	38

1 INTRODUÇÃO

A gestão de frotas é atividade estratégica para organizações que dependem do uso contínuo de veículos em suas operações. Ela envolve simultaneamente três dimensões distintas e interdependentes: a operacional, que trata de quais veículos existem, em que estado se encontram e quais manutenções foram realizadas ou estão previstas; a financeira, que apura quanto cada veículo consome em combustível, peças, serviços, multas, tributos e seguros; e a documental, que acompanha os documentos obrigatórios de veículos e condutores e suas respectivas datas de vencimento.

Essas três dimensões descrevem o mesmo ativo — o veículo — mas, na prática das organizações, costumam ser administradas por pessoas diferentes, em ferramentas diferentes e em momentos diferentes. O setor financeiro registra as despesas em uma planilha; o responsável pela operação anota as manutenções em outro lugar; a documentação permanece arquivada sem qualquer mecanismo de acompanhamento de prazos. O resultado é uma operação em que nenhum ponto isolado da organização enxerga o veículo por inteiro.

Convém observar que o problema não reside na ausência de informação: os dados normalmente existem. Ele decorre da fragmentação dessa informação e da ausência de um mecanismo ativo de acompanhamento de prazos. Uma planilha registra que um documento vence em determinada data, porém não avisa ninguém quando essa data se aproxima, e o vencimento só é percebido quando já produziu consequência.

Nesse contexto, o Fleet Manager é proposto como aplicação web voltada à centralização das informações operacionais, financeiras e documentais de veículos em uma base de dados relacional única. O presente documento descreve o sistema tal como foi efetivamente construído: cada afirmação técnica aqui registrada corresponde a código verificado no repositório do projeto.

1.1 Justificativa

Este trabalho justifica-se pela necessidade de reduzir os problemas decorrentes da gestão de frotas conduzida de maneira manual ou descentralizada. O uso de planilhas isoladas dificulta o acompanhamento preciso de despesas, manutenções e vencimentos, o que compromete a eficiência operacional e amplia riscos financeiros e legais.

A relevância do problema manifesta-se em consequências concretas: multas por documentação vencida, decorrentes do controle passivo de prazos; substituição da

manutenção preventiva pela corretiva, mais onerosa e associada à indisponibilidade do veículo; e custo operacional que permanece invisível, por não ser apurado por ativo. O projeto mostra-se pertinente do ponto de vista acadêmico, por exercitar arquitetura de software, modelagem de dados e segurança de aplicações, e do ponto de vista prático, por endereçar necessidade real de organizações de pequeno e médio porte.

1.2 Objetivo geral

Desenvolver um sistema web para gestão inteligente de frotas, com foco no controle operacional, financeiro e documental de veículos, promovendo maior previsibilidade de custos, redução de riscos legais e suporte à tomada de decisão estratégica.

1.3 Objetivos específicos

- Desenvolver funcionalidades de cadastro, edição e gerenciamento de veículos, motoristas e usuários do sistema, incluindo o vínculo entre motoristas e veículos.
- Permitir o registro de despesas obrigatoriamente vinculadas a um veículo, contemplando as categorias combustível, manutenção, multa, IPVA, seguro e outras.
- Implementar o controle de manutenções preventivas e corretivas, com acompanhamento dos estados programada, realizada e atrasada.
- Registrar documentos obrigatórios de veículos e motoristas, com data de vencimento, classificação automática da situação e anexo do arquivo digital correspondente.
- Disponibilizar painel de indicadores que apresente custo total, custo médio, custo por veículo, evolução mensal das despesas e distribuição por categoria.
- Implementar autenticação de usuários e controle de acesso baseado em papéis, assegurando segregação de responsabilidades no uso do sistema.
- Verificar a qualidade do software por meio de testes automatizados das regras de negócio.

1.4 Estrutura do trabalho

O capítulo 2 apresenta a fundamentação teórica que embasa as decisões adotadas. O capítulo 3 descreve a metodologia de desenvolvimento. O capítulo 4 detalha a construção do sistema, incluindo arquitetura, modelo de dados, segurança e publicação. O capítulo 5 expõe os resultados obtidos e discute as limitações identificadas. O capítulo 6 conclui o trabalho. Os anexos reúnem o termo de abertura do projeto e o documento de arquitetura de software.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo apresenta os fundamentos que embasam o desenvolvimento do Fleet Manager, abordando os conceitos de gestão de frotas, as tecnologias adotadas e os padrões de software empregados na construção do sistema.

2.1 Gestão de frotas

A gestão de frotas compreende o conjunto de atividades voltadas ao planejamento, à organização, ao controle e ao monitoramento dos veículos utilizados por uma organização. O processo envolve o acompanhamento de custos operacionais, a manutenção preventiva e corretiva, o gerenciamento da documentação obrigatória e o monitoramento do desempenho dos condutores. Quando realizada de forma manual ou por meio de planilhas isoladas, essa gestão está sujeita a erros humanos, inconsistências e dificuldades no acompanhamento histórico (FLEURY; WANKE; FIGUEIREDO, 2000).

Sistemas informatizados superam tais limitações ao centralizar as informações e automatizar processos críticos, como a classificação de vencimentos, o cálculo de indicadores financeiros e o controle do estado das manutenções. A informatização reduz o risco de sanções por documentação irregular, melhora a previsibilidade orçamentária e apoia a decisão estratégica.

2.2 Aplicações web e arquitetura cliente-servidor

Sistemas de informação baseados na web são acessados por navegadores e fundamentam-se na arquitetura cliente-servidor, na qual o cliente responde pela interface com o usuário e o servidor pelo processamento das requisições e pela gestão dos dados. Segundo Sommerville (2011), esses sistemas permitem acesso ubíquo e facilitam a manutenção centralizada do software, o que os torna escolha natural para aplicações corporativas.

Categoria relevante de aplicação web é a Single Page Application, que carrega uma única página e atualiza o conteúdo dinamicamente, sem recarregamento integral. A interface do Fleet Manager adota a biblioteca React em conjunto com a linguagem TypeScript, cuja tipagem estática antecipa para o tempo de compilação erros que, de outro modo, só se manifestariam em execução. A construção emprega o Vite e a estilização utiliza o framework utilitário Tailwind CSS. O sistema oferece internacionalização em português brasileiro e inglês por meio da biblioteca i18next.

2.3 Arquitetura REST e interface de programação

REST é um estilo arquitetural para sistemas distribuídos proposto por Fielding (2000), que emprega o protocolo HTTP e seus métodos para operar sobre recursos identificados por localizadores uniformes. Uma interface RESTful promove a separação de responsabilidades entre a camada de apresentação e a de serviço, viabilizando o desenvolvimento independente de cada uma.

No Fleet Manager, a interface de programação é construída em Node.js com o framework Express e organizada em camadas — rotas, middlewares, controladores, serviços e repositórios — com dependência unidirecional. Essa separação não é ornamental: por não conhecerem o protocolo HTTP, os serviços podem ser exercitados por testes automatizados sem que seja necessário iniciar servidor ou banco de dados, o que viabiliza a verificação descrita na seção 5.2.

2.4 Banco de dados relacional e mapeamento objeto-relacional

O PostgreSQL é sistema gerenciador de banco de dados relacional de código aberto, reconhecido pela robustez, pela conformidade com o padrão SQL e pelo suporte a transações com garantias de atomicidade, consistência, isolamento e durabilidade. Tais características asseguram a integridade referencial em cenários com múltiplas entidades relacionadas, como veículos, motoristas, despesas e documentos.

O Prisma é ferramenta de mapeamento objeto-relacional para TypeScript que gera cliente fortemente tipado a partir de um esquema declarativo. Isso reduz erros em tempo de execução e dispensa a escrita manual de SQL nas operações rotineiras. No projeto, o banco é hospedado no Supabase, plataforma que provê PostgreSQL gerenciado, e o Prisma constitui a única via de acesso aos dados a partir da aplicação.

2.5 Autenticação delegada e controle de acesso

A autenticação verifica a identidade de um usuário, ao passo que a autorização determina quais recursos esse usuário pode acessar. São responsabilidades distintas, e o Fleet Manager as atribui a agentes distintos.

A autenticação é delegada ao Supabase Auth. As credenciais deixam de ser responsabilidade da aplicação, que não armazena senha alguma. O serviço emite tokens no padrão JSON Web Token (JONES et al., 2015), assinados com chave assimétrica pelo

algoritmo ES256. A aplicação verifica cada token utilizando a chave pública publicada pelo próprio serviço em endpoint JWKS, conferindo assinatura, emissor e público. Decorre desse desenho uma propriedade relevante: a aplicação não guarda nenhum segredo do serviço de identidade, de modo que não há chave compartilhada a proteger ou rotacionar.

A autorização, por sua vez, permanece sob controle da aplicação e segue o modelo de controle de acesso baseado em papéis. Sandhu et al. (1996) definem o RBAC como mecanismo eficaz para a administração de permissões em sistemas corporativos, por associar permissões a papéis e usuários a papéis, reduzindo a complexidade da administração de segurança. O sistema adota três papéis: administrador, com acesso irrestrito; gestor, responsável pela administração da frota; e operador, com permissões restritas ao registro da rotina operacional.

Cabe destacar uma decisão de projeto: o papel de acesso e a situação de aprovação do usuário são persistidos na base da aplicação, e não transportados no token. O token não constitui, portanto, fonte de autoridade sobre permissões. A consequência prática é que o bloqueio de uma conta produz efeito imediato, sem depender da expiração da credencial em posse do usuário.

2.6 Validação de dados e qualidade de software

A validação dos dados de entrada é prática essencial à integridade e à segurança das informações. O Zod é biblioteca de validação e análise de esquemas para TypeScript que permite declarar, de forma explícita, a estrutura esperada dos dados recebidos. Sua integração ao encadeamento de middlewares do Express possibilita rejeitar requisições malformadas antes que alcancem a camada de negócio.

Quanto à qualidade, os testes automatizados foram implementados com o framework Vitest, que oferece execução rápida e suporte nativo a TypeScript. Adotou-se desenvolvimento orientado a testes nos módulos críticos do servidor. Registre-se, por precisão, que a cobertura concentra-se nas regras de negócio da camada de serviços e nos middlewares de autenticação, autorização e validação; não foram implementados testes de interface nem de integração de ponta a ponta.

3 METODOLOGIA

O desenvolvimento do Fleet Manager baseou-se em abordagem ágil e incremental, com entregas organizadas em ciclos e validação contínua do escopo.

3.1 Ciclo de desenvolvimento

O trabalho foi dividido em etapas incrementais, cada uma dedicada a um conjunto específico de funcionalidades. A etapa inicial tratou da configuração da arquitetura, da padronização de código e da modelagem do banco de dados. As etapas seguintes contemplaram, sucessivamente, autenticação e controle de acesso; cadastro de veículos e motoristas; registro de despesas e manutenções; gestão de documentos com anexo de arquivos; painel de indicadores e administração de usuários; e, por fim, publicação em ambiente de produção.

Ao longo do processo, duas decisões arquiteturais foram revistas com base em evidência prática, e o registro dessas revisões integra o resultado do trabalho. A autenticação, inicialmente delegada a um provedor de identidade externo e em seguida implementada pela própria aplicação, foi por fim atribuída ao Supabase Auth, consolidando autenticação, persistência e armazenamento em um único serviço e eliminando a responsabilidade de guardar senhas. A plataforma de publicação do servidor também foi revista, conforme detalhado na seção 4.10.

3.2 Versionamento e modelagem

O controle de versão foi realizado com Git, e o código hospedado no GitHub. A modelagem precedeu a implementação e incluiu a definição dos casos de uso, do modelo entidade-relacionamento e dos fluxos de autenticação e autorização. O esquema do banco é versionado por migrações incrementais, de modo que a estrutura possa ser reconstruída a partir do repositório em um banco vazio.

3.3 Ferramentas e ambiente

O projeto adota organização em monorepo gerenciado por npm workspaces, reunindo as duas aplicações e um pacote de tipos compartilhados. Optou-se por npm workspaces, e não por ferramentas de orquestração de monorepo mais elaboradas, por adequação ao porte do projeto: o recurso já integra o gerenciador de pacotes e dispensa camada adicional de

construção. A padronização de código emprega ESLint e Prettier. A Tabela 1 sintetiza as tecnologias adotadas em cada camada.

Tabela 1 — Tecnologias adotadas por camada

Camada	Tecnologia
Linguagem	TypeScript, no servidor e na interface
Interface	React 18, Vite, React Router, Tailwind CSS, Recharts, i18next
Servidor	Node.js com Express 4, em arquitetura de camadas
Mapeamento objeto-relacional	Prisma 6
Banco de dados	PostgreSQL gerenciado pelo Supabase
Autenticação	Supabase Auth, com verificação de token por JWKS
Armazenamento de arquivos	Supabase Storage
Validação	Zod
Agendamento	node-cron em ambiente local; agendador da plataforma em produção
Testes	Vitest
Organização do repositório	Monorepo com npm workspaces
Publicação	Vercel, em projeto único

Fonte: elaborado pelos autores (2026).

4 DESENVOLVIMENTO

Este capítulo descreve a construção do sistema. A Figura 1 apresenta a visão geral da solução e situa os componentes descritos nas seções seguintes.

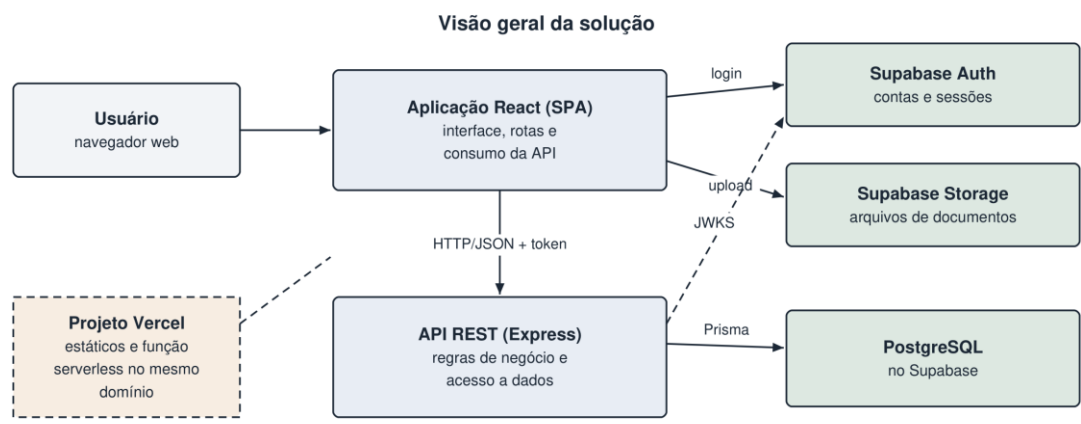


Figura 1 — Visão geral da solução

Fonte: elaborado pelos autores (2026).

O Supabase cumpre três papéis independentes: autentica os usuários, hospeda o banco relacional e armazena os arquivos anexados aos documentos. Nem a autenticação nem o envio de arquivos transitam pela camada de serviço — implicações discutidas nas seções 4.5 e 4.7.

4.1 Casos de uso

A Figura 2 apresenta os casos de uso significativos para a arquitetura, organizados por papel. Os papéis são cumulativos: o gestor exerce os casos do operador, e o administrador exerce os do gestor, acrescidos da administração de usuários.

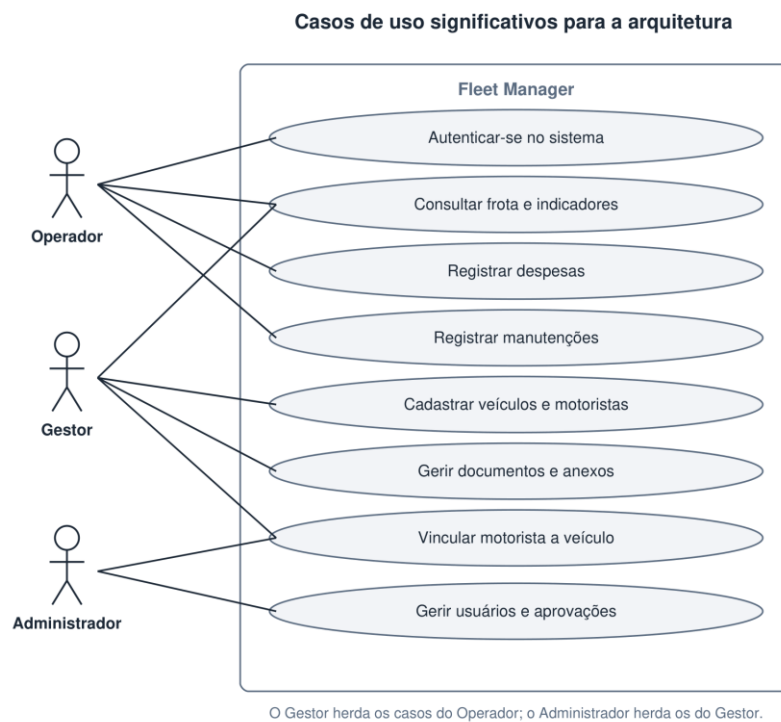


Figura 2 — Casos de uso significativos para a arquitetura

Fonte: elaborado pelos autores (2026).

4.2 Arquitetura em camadas

O servidor adota arquitetura em camadas com dependência unidirecional: cada camada conhece apenas a imediatamente inferior. A Figura 3 apresenta essa organização.

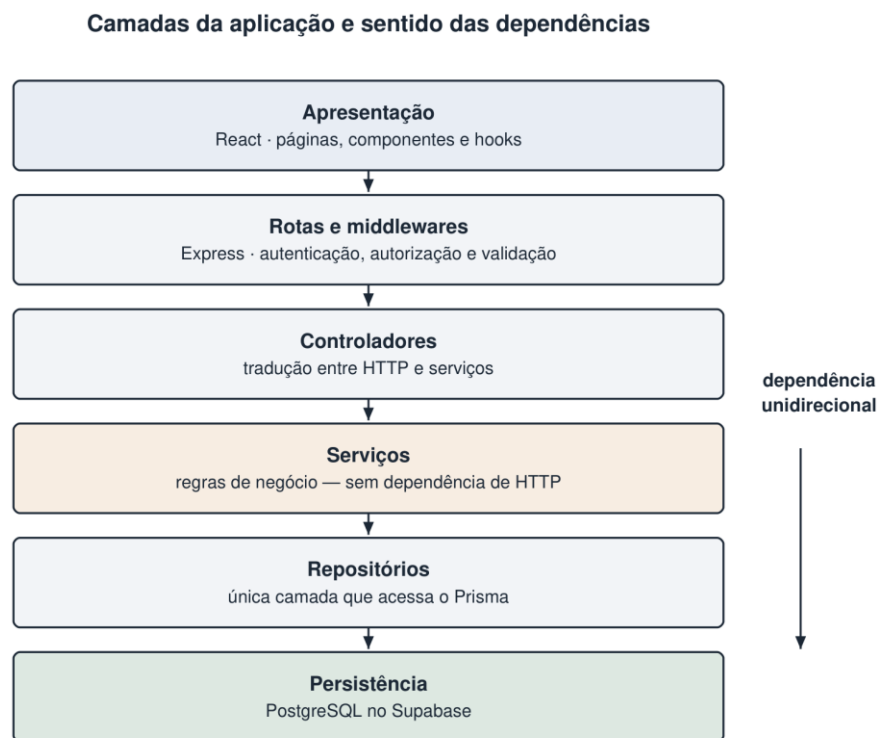


Figura 3 — Camadas da aplicação e sentido das dependências

Fonte: elaborado pelos autores (2026).

A responsabilidade de cada camada é delimitada de forma estrita. As rotas declaram os caminhos, definem o esquema de validação e encadeiam os middlewares, sem conter lógica. Os controladores traduzem a requisição HTTP em chamada de serviço e o retorno em resposta HTTP, sem conter regra de negócio. Os serviços concentram as regras de negócio e não conhecem o Express: não recebem os objetos de requisição e resposta. Os repositórios encapsulam todo o acesso ao banco e constituem a única camada que importa o cliente Prisma.

O isolamento da camada de serviços em relação ao protocolo HTTP é o que viabiliza os testes automatizados descritos na seção 5.2: os serviços são exercitados com repositórios substituídos por dublês, sem necessidade de servidor ou banco em execução.

4.3 Organização do repositório

O código é organizado em monorepo, conforme a Figura 4. A escolha justifica-se pelo compartilhamento de tipos entre a interface e o servidor: uma alteração em uma enumeração provoca erro de compilação em ambos os lados, o que impede a divergência silenciosa do contrato de dados.

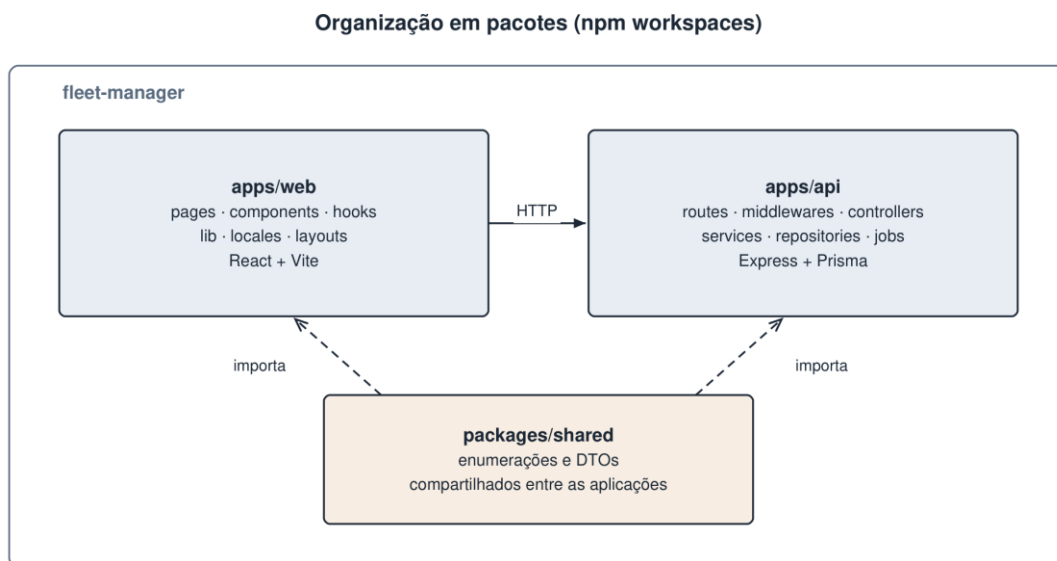


Figura 4 — Organização em pacotes do monorepo

Fonte: elaborado pelos autores (2026).

O pacote compartilhado publica suas definições em dois formatos. A exigência decorre dos consumidores: o ambiente de execução do servidor requer módulos CommonJS, ao passo que o empacotador da interface exige módulos ECMAScript para resolver importações nomeadas. A compilação ocorre automaticamente após a instalação das dependências.

4.4 Modelo de dados

O modelo é composto por seis entidades e oito enumerações, organizadas em torno da entidade Veículo, que concentra as três dimensões da gestão de frota. A Figura 5 apresenta o modelo entidade-relacionamento.

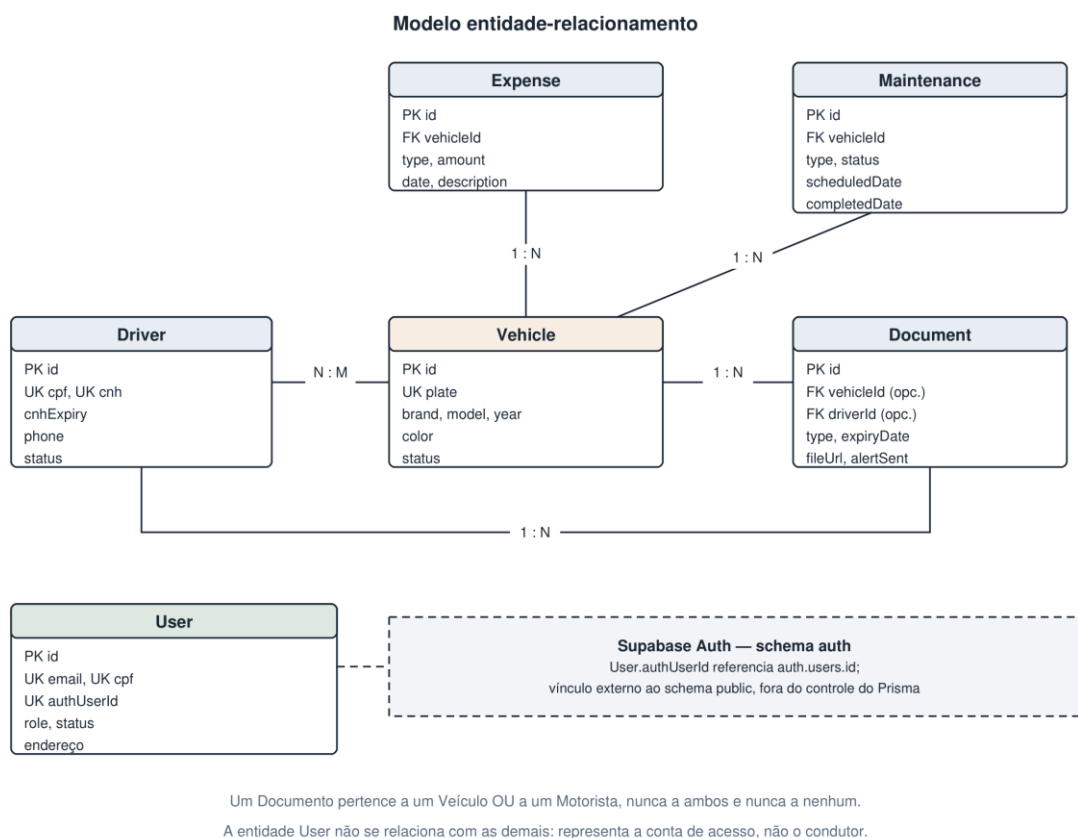


Figura 5 — Modelo entidade-relacionamento

Fonte: elaborado pelos autores (2026).

Tabela 7 — Entidades do modelo de dados

Entidade	Finalidade e observações
Vehicle	Veículo da frota. Entidade central; a placa é única.
Driver	Motorista, com número e validade da habilitação. CPF e CNH únicos.
Expense	Despesa operacional, obrigatoriamente vinculada a um veículo. O valor usa tipo decimal, evitando erro de arredondamento.
Maintenance	Manutenção preventiva ou corretiva, com data prevista e de conclusão.
Document	Documento obrigatório com data de vencimento e anexo digital opcional. Vincula-se a um veículo ou a um motorista.
User	Perfil de acesso: dados cadastrais, papel e situação de aprovação. As credenciais residem no serviço de autenticação.

Fonte: elaborado pelos autores (2026).

Três aspectos do modelo merecem registro. Primeiro, a obrigatoriedade do vínculo entre despesa e veículo, garantida por restrição no banco, é o que torna confiável a apuração do custo por ativo. Segundo, a situação de vencimento do documento não é armazenada: ela é calculada a cada consulta a partir da data de validade, o que elimina a possibilidade de o dado ficar defasado. Terceiro, as exclusões em cascata sustentam a distinção entre desativar um registro, que preserva o histórico, e excluí-lo permanentemente.

Registra-se, como limitação do modelo, que as entidades Usuário e Motorista não possuem vínculo formal, ainda que o papel de operador corresponda conceitualmente ao condutor. A unificação está prevista como evolução.

4.5 Autenticação e controle de acesso

A Figura 6 apresenta o fluxo de autenticação e o acesso subsequente a um recurso protegido.

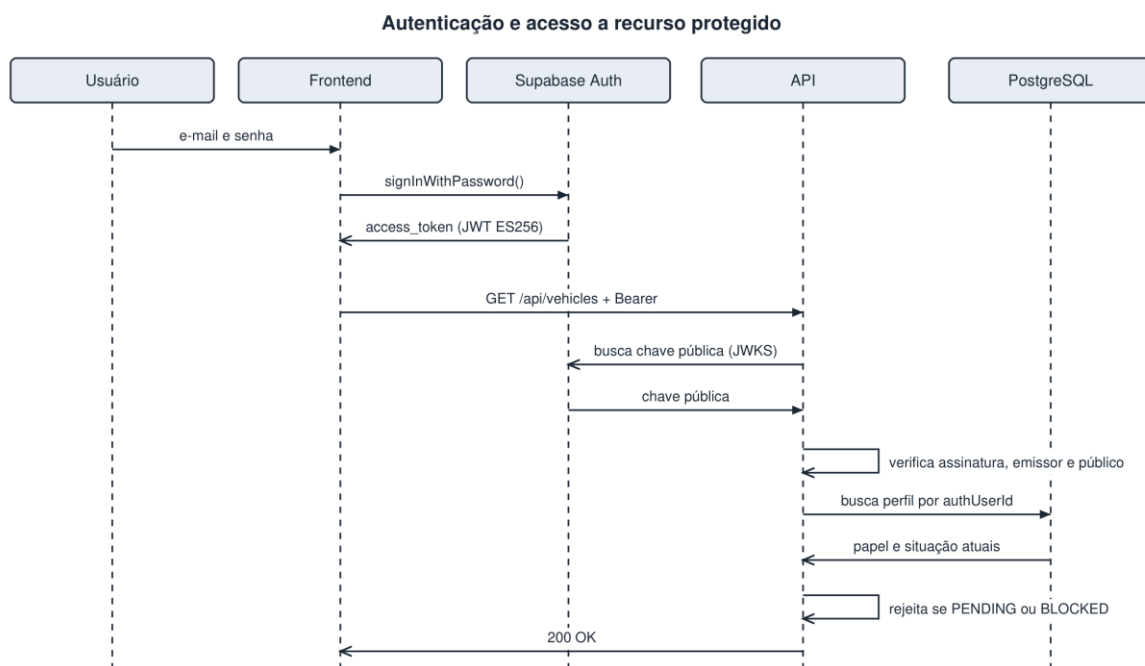


Figura 6 — Autenticação e acesso a recurso protegido

Fonte: elaborado pelos autores (2026).

O cadastro ocorre em duas etapas. A interface cria a conta no serviço de autenticação e obtém a sessão; de posse do token, envia os dados cadastrais à aplicação, que cria o perfil com situação pendente. Quando já existe perfil com o mesmo endereço de correio eletrônico e sem conta vinculada, a aplicação associa o perfil existente em vez de criar outro, preservando papel e situação. Caso a conta esteja autenticada mas ainda não possua perfil, a aplicação responde com código específico e a interface conduz o usuário à conclusão do cadastro.

A Figura 9 apresenta o ciclo de vida da conta. Contas recém-criadas permanecem pendentes até que um administrador as aprove, e o bloqueio produz efeito imediato, pois o perfil é reconsultado a cada requisição.



Figura 9 — Ciclo de vida da conta de usuário

Fonte: elaborado pelos autores (2026).

A Tabela 6 apresenta a matriz de controle de acesso efetivamente aplicada, extraída da configuração das rotas.

Tabela 6 — Matriz de controle de acesso por papel

Recurso ou operação	Administrador	Gestor	Operador
Gerir usuários e aprovações	Sim	Não	Não
Editar o próprio perfil	Sim	Sim	Sim
Consultar veículos e motoristas	Sim	Sim	Sim
Cadastrar e editar veículos	Sim	Sim	Não
Cadastrar e editar motoristas	Sim	Sim	Não
Vincular motorista a veículo	Sim	Sim	Não
Registrar despesas e manutenções	Sim	Sim	Sim
Cadastrar e editar documentos	Sim	Sim	Não
Excluir registros	Sim	Sim	Não
Consultar o painel de indicadores	Sim	Sim	Sim

Fonte: elaborado pelos autores (2026).

4.6 Módulos funcionais

O sistema compreende oito módulos. O de veículos oferece cadastro, edição, consulta com filtros, desativação, reativação e exclusão permanente. O de motoristas segue estrutura equivalente e acrescenta o controle da validade da habilitação. O vínculo entre ambos é de muitos para muitos, com tela dedicada.

O módulo de despesas registra lançamentos obrigatoriamente vinculados a um veículo, distribuídos em seis categorias, com filtros por veículo, categoria e período. O de manutenções distingue as preventivas das corretivas e acompanha três estados, mantendo a coerência entre estado e data de conclusão. O de documentos registra a data de vencimento, classifica automaticamente a situação e permite anexar arquivo digital.

A central de alertas consolida vencimentos e pendências, e contadores na interface indicam o volume de itens que exigem atenção. O painel de indicadores apura custo total,

custo médio, quantidade de lançamentos, evolução mensal, distribuição por categoria e custo por veículo, com filtros por período, veículo e categoria. A administração de usuários permite listar contas, alterar papel e situação e excluir registros.

4.7 Armazenamento de arquivos

O anexo de arquivos aos documentos utiliza o serviço de armazenamento do Supabase, com envio realizado diretamente pelo navegador, conforme a Figura 7.

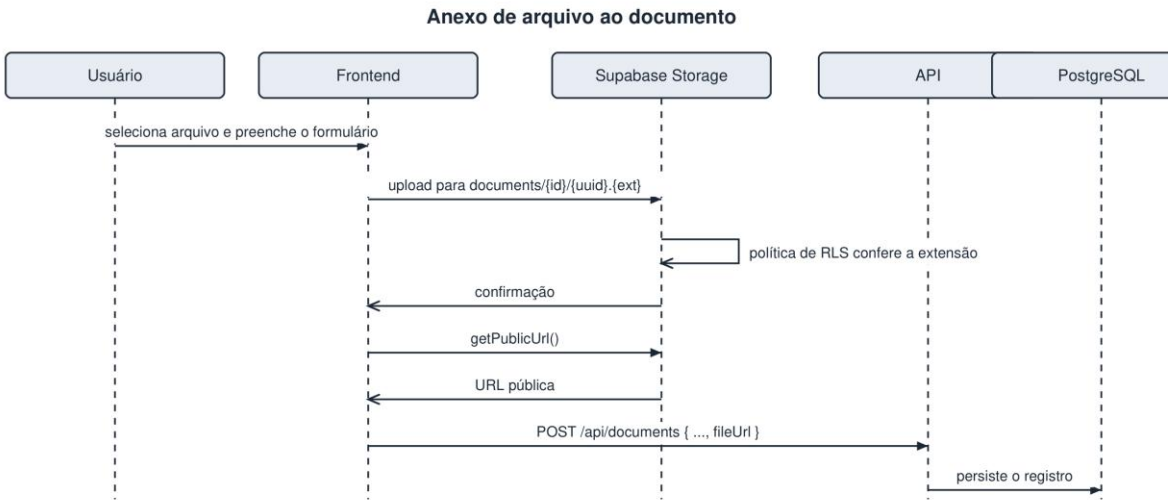


Figura 7 — Anexo de arquivo ao documento

Fonte: elaborado pelos autores (2026).

O desenho reduz a carga sobre a camada de serviço e dispensa o tráfego do arquivo por um intermediário. Em contrapartida, introduz limitações que devem ser declaradas: a validação do tipo de arquivo é realizada pela política de acesso do próprio serviço de armazenamento, e não pela aplicação; a exclusão de um documento remove o registro no banco, mas não o arquivo correspondente; e o repositório de arquivos é público, de modo que quem detiver o endereço acessa o conteúdo sem autenticação.

4.8 Rotina automatizada de vencimentos

Uma rotina diária identifica os documentos que vencem em até trinta dias e os sinaliza para acompanhamento, de forma idempotente: documentos já sinalizados não são reprocessados, e execuções repetidas no mesmo dia não produzem efeito colateral.

Delimita-se explicitamente o alcance dessa rotina: ela não envia notificação para fora do sistema. O acompanhamento de prazos é realizado dentro da aplicação, pela central de alertas e pelos indicadores de vencimento. A notificação por canais externos foi avaliada e excluída do escopo, conforme a seção 4.11.

4.9 Proteção do banco de dados

A autorização é aplicada na camada de aplicação, e o banco é acessado exclusivamente pelo servidor. Ocorre, porém, que a plataforma de hospedagem expõe automaticamente o esquema público por uma interface REST gerada, acessível com a chave pública do projeto — chave que, por natureza, é distribuída junto com a interface. Sem proteção, as tabelas seriam legíveis e graváveis por quem a obtivesse, contornando integralmente a aplicação e o controle de acesso por papel.

Adotaram-se, portanto, duas camadas de proteção. A primeira habilita a segurança em nível de linha em todas as tabelas, sem criar política alguma, o que nega acesso aos papéis públicos; o papel proprietário das tabelas não é submetido a essa restrição, de modo que a aplicação opera normalmente. A segunda revoga os privilégios desses papéis sobre o esquema, inclusive para tabelas criadas futuramente. A ausência de políticas é intencional: elas só fariam sentido se o cliente consultasse o banco diretamente, o que esta arquitetura não faz. O objetivo é fechar um caminho de acesso que a plataforma abre por padrão, e não replicar a autorização no banco.

4.10 Publicação em produção

O sistema é publicado como projeto único, com a interface estática e a camada de serviço atendidas pelo mesmo domínio, conforme a Figura 8.

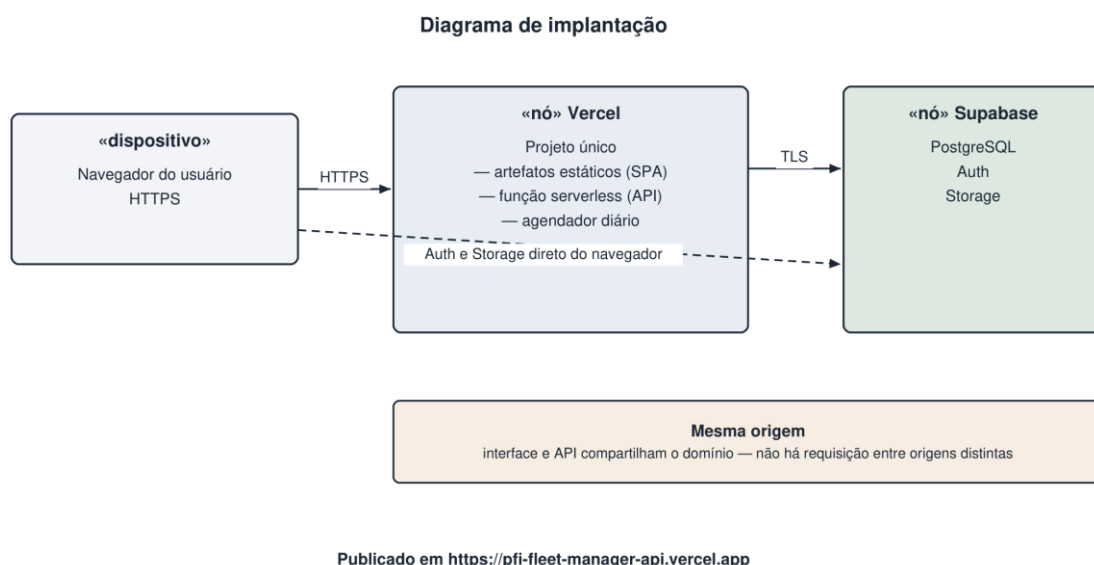


Figura 8 — Diagrama de implantação

Fonte: elaborado pelos autores (2026).

A decisão pelo domínio compartilhado tem consequência direta: não há requisição entre origens distintas entre a interface e a camada de serviço, o que elimina toda uma classe de configuração e de falhas. A plataforma inicialmente prevista para o servidor foi substituída após verificar-se que não oferece mais camada gratuita permanente, condição necessária à continuidade do projeto até a apresentação.

A adaptação ao modelo de execução sem servidor dedicado impôs dois ajustes ao código. O primeiro decorre da ausência de processo persistente entre requisições: o agendamento interno da rotina de vencimentos é desativado em produção, e a execução passa a ser disparada pelo agendador da plataforma, por meio de rota protegida por segredo. O segundo refere-se ao pacote compartilhado, cuja publicação em código-fonte TypeScript não é carregável pelo ambiente de execução, o que motivou a compilação descrita na seção 4.3.

4.11 Delimitação do escopo

Não integram o escopo deste trabalho o rastreamento por satélite em tempo real, a telemetria avançada, o planejamento e a otimização de rotas, a integração automática com sistemas governamentais e o desenvolvimento de aplicativo móvel nativo. A solução é entregue como aplicação web responsiva, acessível por navegador em dispositivos móveis.

Excluiu-se igualmente a notificação de vencimentos por correio eletrônico, mensagem de texto ou canal equivalente. Tal recurso exigiria a contratação e a configuração de serviço

externo de envio, com custo e dependência operacional não justificáveis no contexto do trabalho. O acompanhamento de prazos é realizado dentro da aplicação, o que atende ao propósito de substituir o controle passivo por acompanhamento ativo sem introduzir dependência de infraestrutura de terceiros. Como consequência dessa exclusão, não se implementou a recuperação autônoma de senha, que depende do mesmo canal; a alteração de senha permanece disponível ao usuário autenticado.

5 RESULTADOS E DISCUSSÕES

Esta seção apresenta o que foi efetivamente entregue, a verificação realizada sobre o código e as limitações identificadas.

5.1 Requisitos atendidos

Os dez requisitos funcionais definidos no termo de abertura foram atendidos. A Tabela 3 relaciona cada requisito à sua situação e à evidência correspondente no sistema.

Tabela 3 — Requisitos funcionais e situação de atendimento

Código	Requisito	Situação
RF01	Cadastro, edição e exclusão de veículos	Atendido
RF02	Cadastro e gerenciamento de motoristas	Atendido
RF03	Despesas vinculadas obrigatoriamente a um veículo	Atendido
RF04	Registro de diferentes categorias de despesa	Atendido
RF05	Controle de manutenções preventivas e corretivas	Atendido
RF06	Registro de documentos com data de vencimento	Atendido
RF07	Emissão de alertas de vencimento	Atendido dentro da aplicação
RF08	Indicadores financeiros por veículo e por período	Atendido
RF09	Autenticação de usuários	Atendido
RF10	Controle de acesso por papel	Atendido

Fonte: elaborado pelos autores (2026).

O RF07 merece esclarecimento quanto ao canal de emissão. O sistema classifica automaticamente documentos e manutenções por situação de vencimento, consolida-os em uma central de alertas e exibe contadores na interface, além de executar diariamente a rotina de sinalização. A notificação por canais externos, contudo, foi excluída do escopo, conforme a seção 4.11. O requisito é, portanto, atendido no sentido da exibição, e não no da notificação ativa fora do sistema — delimitação declarada de forma explícita para que o alcance não seja interpretado de maneira ampliada.

Além dos requisitos previstos no levantamento inicial, o desenvolvimento incorporou funcionalidades identificadas durante o trabalho: vínculo de muitos para muitos entre veículos e motoristas; anexo de arquivo digital aos documentos, com visualização; fluxo de aprovação de contas por administrador; bloqueio e reativação de contas; edição do perfil próprio; distinção entre desativação e exclusão permanente; e interface em português brasileiro e inglês.

Tabela 4 — Requisitos não funcionais e situação de atendimento

Código	Requisito	Situação
RNF01	Aplicação web responsiva	Atendido
RNF02	Armazenamento seguro em banco relacional	Atendido

RNF03	Controle de autenticação e autorização	Atendido
RNF04	Tempo de resposta adequado nas operações comuns	Sem medição formal
RNF05	Arquitetura cliente-servidor	Atendido
RNF06	Escalabilidade para novas funcionalidades	Atendido
RNF07	Integridade e consistência das informações	Atendido

Fonte: elaborado pelos autores (2026).

Quanto ao RNF04, as consultas empregam índices e agregações realizadas no próprio banco, e o comportamento observado em uso é compatível com o limite estabelecido. Não foi, entretanto, conduzida medição formal de desempenho, razão pela qual o atendimento não se considera comprovado por evidência empírica.

Tabela 5 — Regras de negócio implementadas

Código	Regra
RN01	Toda despesa está obrigatoriamente vinculada a um veículo.
RN02	Toda manutenção está obrigatoriamente vinculada a um veículo.
RN03	Um documento vincula-se a um veículo ou a um motorista, nunca a ambos e nunca a nenhum.
RN04	Contas recém-criadas permanecem pendentes até aprovação por um administrador.
RN05	Contas bloqueadas têm o acesso negado de imediato, mesmo portando token válido.
RN06	Placa, CPF, habilitação e endereço de correio eletrônico são únicos.
RN07	A exclusão permanente de veículo ou motorista remove em cascata os registros dependentes.
RN08	A desativação preserva o registro e seu histórico.
RN09	A data de conclusão da manutenção é mantida coerente com o estado.
RN10	A situação de vencimento é calculada dinamicamente, não armazenada.
RN11	Documentos que vencem em até trinta dias são sinalizados diariamente, de forma idempotente.
RN12	Os dados de identificação do veículo são normalizados em maiúsculas.

Fonte: elaborado pelos autores (2026).

5.2 Verificação da qualidade

A qualidade do software foi verificada por meio de execução automatizada sobre o código do repositório. A Tabela 8 apresenta os resultados obtidos.

Tabela 8 — Verificações executadas sobre o código

Verificação	Resultado
Compilação TypeScript do servidor	Sem erros
Compilação TypeScript da interface	Sem erros
Análise estática com ESLint	Sem erros
Testes automatizados com Vitest	100 testes aprovados, em 13 arquivos
Construção do pacote de produção	Concluída com sucesso
Publicação em produção	Concluída e verificada em ambiente publicado

Fonte: elaborado pelos autores (2026).

Os cem testes distribuem-se entre os serviços de veículos, motoristas, despesas, manutenções, documentos, indicadores, usuários e autenticação, além dos middlewares de autenticação, autorização e validação. Reitera-se que a cobertura concentra-se nas regras de negócio: não há testes de interface nem de integração de ponta a ponta.

A verificação do ambiente publicado abrangeu o carregamento da interface, o roteamento da aplicação de página única, a rejeição de requisições sem credencial, o fluxo completo de autenticação, a consulta a dados reais, o painel de indicadores e a rotina agendada, inclusive quanto à recusa de chamadas sem o segredo correspondente.

5.3 Limitações e discussão

A principal limitação do trabalho é de natureza metodológica: o sistema não foi aplicado em uma organização real. A validação empregou base de dados fictícia, ainda que consistente e gerada por rotina automatizada. Em consequência, os benefícios pretendidos — redução de sanções por documentação vencida, migração da manutenção corretiva para a preventiva e visibilidade do custo por veículo — permanecem como expectativas fundamentadas, e não como resultados medidos. A comprovação empírica demandaria adoção por uma frota real, com medição comparativa antes e depois, o que se situa fora do escopo temporal deste trabalho.

Do ponto de vista técnico, registram-se as seguintes limitações. A validação do tipo de arquivo anexado ocorre na política do serviço de armazenamento, e não na camada de aplicação. A exclusão de um documento não remove o arquivo correspondente, o que permite o acúmulo de arquivos órfãos. O repositório de arquivos é público, de modo que o conhecimento do endereço basta para o acesso. As entidades Usuário e Motorista não possuem vínculo formal no modelo de dados. Não há registro de autoria dos lançamentos, o que impede determinar qual usuário registrou determinada despesa. E a verificação de posse do endereço de correio eletrônico está desativada no cadastro, decorrência da exclusão do envio de mensagens do escopo; o risco é mitigado pela aprovação manual exigida antes de qualquer acesso.

Cada uma dessas limitações foi identificada por verificação do próprio código, e não inferida. O registro explícito cumpre função metodológica: delimita o que o sistema efetivamente entrega e distingue-o do que permanece como evolução possível.

6 CONCLUSÃO

Este trabalho apresentou o Fleet Manager, sistema web para gestão de frotas que centraliza o controle operacional, financeiro e documental de veículos em uma base de dados relacional única. O objetivo geral foi alcançado: o sistema foi desenvolvido, verificado e publicado em ambiente de produção, e os dez requisitos funcionais definidos no termo de abertura foram atendidos.

A contribuição técnica do trabalho concentra-se em três decisões arquiteturais. A primeira é a organização do servidor em camadas com dependência unidirecional, que não se limita a uma escolha estética: por isolarem-se do protocolo HTTP, as regras de negócio puderam ser exercitadas por cem testes automatizados sem infraestrutura de apoio. A segunda é a delegação da autenticação a serviço especializado com verificação por chave pública, mantendo, contudo, a autorização sob controle da aplicação — desenho que retira do sistema a responsabilidade de guardar senhas sem transferir a terceiros a autoridade sobre permissões. A terceira é o compartilhamento de tipos entre interface e servidor, que converte divergências de contrato em erros de compilação.

O percurso também produziu aprendizado sobre a distância entre projetar e publicar. Duas exigências só se manifestaram na publicação: o pacote compartilhado precisou passar a emitir código compilado, pois o ambiente de execução não interpreta TypeScript, e a rotina agendada precisou migrar do processo da aplicação para o agendador da plataforma, ausente o processo persistente. Ambas as adaptações foram incorporadas ao sistema e registradas neste documento por constituírem resultado legítimo do trabalho.

Como delimitação, reitera-se que o sistema não foi aplicado em organização real, razão pela qual os benefícios pretendidos permanecem como expectativas fundamentadas. A continuidade natural do trabalho compreende a validação em frota real, com medição comparativa; a unificação das entidades Usuário e Motorista; o registro de autoria dos lançamentos; e a ampliação da cobertura de testes para as camadas de integração e interface.

REFERÊNCIAS

FIELDING, Roy Thomas. Architectural Styles and the Design of Network-based Software Architectures. 2000. Tese (Doutorado em Ciência da Computação) — University of California, Irvine, 2000.

FLEURY, Paulo Fernando; WANKE, Peter; FIGUEIREDO, Kleber Fossati (org.). Logística empresarial: a perspectiva brasileira. São Paulo: Atlas, 2000.

JONES, Michael B. et al. JSON Web Token (JWT). RFC 7519. Internet Engineering Task Force, maio 2015. Disponível em: <https://datatracker.ietf.org/doc/html/rfc7519>. Acesso em: 26 ago. 2026.

KRUCHTEN, Philippe. The "4+1" View Model of Software Architecture. IEEE Software, v. 12, n. 6, p. 42-50, nov. 1995.

META. React: The library for web and native user interfaces. Meta Open Source, 2026. Disponível em: <https://react.dev>. Acesso em: 26 ago. 2026.

MICROSOFT. TypeScript Documentation. 2026. Disponível em: <https://www.typescriptlang.org/docs>. Acesso em: 26 ago. 2026.

OPENJS FOUNDATION. Node.js Documentation. 2026. Disponível em: <https://nodejs.org/docs>. Acesso em: 26 ago. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL Documentation. 2026. Disponível em: <https://www.postgresql.org/docs>. Acesso em: 26 ago. 2026.

PRISMA DATA, INC. Prisma Documentation: next-generation ORM for Node.js and TypeScript. 2026. Disponível em: <https://www.prisma.io/docs>. Acesso em: 26 ago. 2026.

SANDHU, Ravi S. et al. Role-based access control models. IEEE Computer, v. 29, n. 2, p. 38-47, fev. 1996.

SOMMERVILLE, Ian. Engenharia de Software. 9. ed. São Paulo: Pearson Prentice Hall, 2011.

SUPABASE. Supabase Documentation. 2026. Disponível em: <https://supabase.com/docs>. Acesso em: 26 ago. 2026.

TAILWIND LABS. Tailwind CSS Documentation. 2026. Disponível em: <https://tailwindcss.com/docs>. Acesso em: 26 ago. 2026.

VERCEL, INC. Vercel Documentation. 2026. Disponível em: <https://vercel.com/docs>. Acesso em: 26 ago. 2026.

VITEST. Vitest: a Vite-native testing framework. 2026. Disponível em: <https://vitest.dev>. Acesso em: 26 ago. 2026.

VITE TEAM. Vite Documentation. 2026. Disponível em: <https://vite.dev>. Acesso em: 26 ago. 2026.

ZOD. Zod: TypeScript-first schema validation with static type inference. 2026. Disponível em: <https://zod.dev>. Acesso em: 26 ago. 2026.

ANEXO A — TERMO DE ABERTURA DO PROJETO

A.1 Identificação do projeto

Nome do projeto: Fleet Manager — Sistema de Gestão Inteligente de Frotas. Código do projeto: FM-PFI-2026.

A.2 Objetivos

Desenvolver um sistema web para gestão de frotas, com foco no controle operacional, financeiro e documental de veículos, permitindo maior previsibilidade de custos, redução de riscos legais e apoio à tomada de decisão estratégica.

A.3 Escopo

O projeto contempla o cadastro e a gestão de veículos e motoristas; o registro de despesas operacionais vinculadas a cada veículo; o controle de manutenções preventivas e corretivas, com acompanhamento de estado; o controle de documentos obrigatórios, com classificação de vencimentos; o painel de indicadores financeiros; e os mecanismos de autenticação e controle de acesso por papel.

Não fazem parte do escopo o rastreamento por satélite em tempo real, a telemetria avançada, o planejamento e a otimização de rotas, a integração automática com sistemas governamentais, o desenvolvimento de aplicativo móvel nativo e a notificação de vencimentos por canais externos.

A.4 Cronograma resumido

Tabela 9 — Cronograma resumido

Fase	Marco	Período
Iniciação	Projeto aprovado	Fevereiro de 2026
Planejamento	Requisitos e arquitetura definidos	Março de 2026
Execução	Servidor e modelo de dados concluídos	Abril de 2026
Execução	Interface e módulos funcionais concluídos	Maio de 2026
Execução	Painel de indicadores e gestão de usuários	Junho de 2026
Monitoramento	Produto mínimo viável verificado	Junho de 2026
Execução	Migração da autenticação e publicação em produção	Agosto de 2026
Encerramento	Entrega e apresentação	Agosto de 2026

Fonte: elaborado pelos autores (2026).

A.5 Partes interessadas

Tabela 10 — Partes interessadas do projeto

Participante	Responsabilidade	Autoridade
Professor orientador	Orientação, acompanhamento e avaliação do projeto	Alta
André Luiz Cavalcante da Silva	Desenvolvimento, modelagem e documentação	Alta
Luiz Eduardo Pacheco	Desenvolvimento, modelagem e documentação	Alta
Gregory Figueiredo de M. P. Jereissati	Desenvolvimento, modelagem e documentação	Alta
Gestores de frota (usuários finais)	Utilização estratégica e validação do sistema	Média
Operadores (usuários finais)	Registro e atualização das informações operacionais	Baixa

Fonte: elaborado pelos autores (2026).

A.6 Restrições

- Orçamento nulo: o projeto restringe-se a serviços com camada gratuita permanente, condição que determinou a escolha das plataformas de hospedagem.
- Equipe de três integrantes, em regime acadêmico, com dedicação parcial.
- Prazo delimitado pelo calendário do Projeto Final Integrador I.
- Ausência de acesso a uma frota real para validação empírica.

A.7 Riscos e mitigação

- Dependência de plataformas de terceiros. Materializou-se durante o projeto: o ambiente de banco de dados foi perdido e precisou ser integralmente recriado. Mitigação adotada: versionamento do esquema por migrações e de scripts de configuração, tornando o ambiente reconstruível a partir do repositório.
- Alteração das condições comerciais dos serviços utilizados. Materializou-se na plataforma inicialmente prevista para o servidor, que deixou de oferecer camada gratuita permanente. Mitigação adotada: substituição por alternativa gratuita.
- Divergência entre a documentação e o sistema implementado. Mitigação adotada: verificação de cada afirmação técnica contra o código-fonte antes do registro.
- Concentração de conhecimento em um integrante. Mitigação adotada: revisão conjunta e documentação das decisões arquiteturais.

A.8 Critérios de sucesso

- Atendimento dos dez requisitos funcionais definidos no levantamento.
- Sistema publicado e operante em ambiente de produção, acessível por navegador.

- Compilação sem erros e aprovação integral dos testes automatizados das regras de negócio.
- Ambiente reconstruível a partir do repositório, sem dependência de configuração não documentada.
- Documentação técnica correspondente ao sistema efetivamente implementado.

ANEXO B — DOCUMENTO DE ARQUITETURA DE SOFTWARE

Este anexo descreve as decisões arquiteturais do Fleet Manager segundo o modelo de visões "4+1" (KRUCHTEN, 1995), que organiza a descrição de uma arquitetura em visões complementares, cada uma dirigida a um público distinto.

B.1 Requisitos e restrições arquiteturais

Tabela 2 — Requisitos e restrições arquiteturais

Requisito	Solução adotada
Linguagem	TypeScript no servidor e na interface, com tipos compartilhados entre ambos.
Plataforma	Interface estática e camada de serviço publicadas em projeto único na Vercel, no mesmo domínio.
Segurança	Autenticação delegada ao Supabase Auth, com token ES256 verificado pela chave pública do serviço; autorização por papéis aplicada na aplicação.
Persistência	PostgreSQL gerenciado pelo Supabase, acessado exclusivamente pelo Prisma.
Armazenamento de arquivos	Supabase Storage, com envio direto pelo navegador e política de restrição de tipos.
Internacionalização	Português brasileiro e inglês, com preferência persistida no navegador.
Custo	Restrição a serviços com camada gratuita permanente.

Fonte: elaborado pelos autores (2026).

B.2 Visão de casos de uso

Apresentada na Figura 2, seção 4.1. Os casos significativos para a arquitetura são a autenticação, que atravessa todas as rotas protegidas, e a gestão de usuários, que determina o papel aplicado nas demais operações.

B.3 Visão lógica

Apresentada na Figura 3, seção 4.2. A aplicação organiza-se em camadas com dependência unidirecional. As classes mais significativas concentram-se na camada de serviços, que materializa as regras de negócio descritas na Tabela 5, e na de repositórios, que isola o acesso ao banco. Os middlewares de autenticação, autorização e validação atuam de forma transversal às rotas protegidas.

B.4 Visão de implementação

Apresentada na Figura 4, seção 4.3. O código distribui-se em duas aplicações e um pacote compartilhado, este último contendo as enumerações e os objetos de transferência de

dados utilizados por ambas. A Figura 6 detalha a realização do caso de uso de autenticação, e a Figura 7, a do anexo de arquivo a documento.

B.5 Visão de processo

O sistema apresenta dois fluxos de execução distintos. O primeiro é síncrono e orientado a requisição: cada chamada é atendida de forma independente, sem estado compartilhado entre requisições, o que se coaduna com o modelo de execução sem servidor dedicado adotado na publicação. O segundo é a rotina diária de sinalização de vencimentos, disparada pelo agendador da plataforma e projetada para ser idempotente, de modo que execuções repetidas não produzam efeito colateral.

B.6 Visão de implantação

Apresentada na Figura 8, seção 4.10. A topologia compreende três nós: o dispositivo do usuário, executando o navegador; o projeto na plataforma de nuvem, que atende tanto os artefatos estáticos quanto a camada de serviço; e a plataforma de dados, que provê banco relacional, autenticação e armazenamento de arquivos.

B.7 Visão de dados

Apresentada na Figura 5, seção 4.4, e detalhada na Tabela 7. O esquema é versionado por migrações incrementais, o que permite reconstruir integralmente a estrutura relacional a partir do repositório. A proteção do banco contra acesso externo é descrita na seção 4.9.